

KTH Electrical Engineering and Computer Science

Assignment 1

CUDA Vector Addition and Matrix Multiplication and Quantum Fourier Transform Profiling

Applied GPU Programming · HT26

Group 40

Yuanyu Zhang · Hongen Wang · Xinyang Li

Stockholm, 6 November 2026

Experimental environments 1

Question 1 — CUDA Vector Addition 2

Question 2 — 2D Dense Matrix Multiplication 4

Question 3 — Quantum Fourier Transformation 7

Experimental environments

Question	GPU and system	CUDA and profiler	Purpose
Q1	Tesla T4, Google Colab	CUDA 13.0.88, Nsight Compute 2025.3.1	Timing, correctness and occupancy
Q2	Tesla T4, Google Colab	CUDA 13.0.88, Nsight Compute 2025.3.1	Timing, correctness and occupancy
Q3	Tesla T4, Google Colab	QuEST CUDA backend, Nsight Systems	QFT runtime and memory profiling

All three questions use an NVIDIA T4 with compute capability 7.5. All Q2 timings are ordinary runs without profiler instrumentation. Each Q1/Q2 program runs one kernel per data point, without averaging repeated measurements.

Question 1 — Your First CUDA Program and GPU Performance Metrics

1. Floating-point operations and global-memory reads

For a vector of length N, the kernel launches enough threads for one thread per element. Each valid thread performs one floating-point addition and reads one `float` from each input vector. Therefore,

$$\text{FLOPs} = N, \qquad \text{global-memory reads} = 2N.$$

The kernel also performs N global-memory writes to the output vector.

2–3. Configuration and occupancy for $N = 512$

The block size is 32 threads. For $N = 512$, the launch uses $512/32 = 16$ blocks, hence 512 threads in total. Nsight Compute reports 3.12% achieved occupancy.

4. Correctness for $N = 263149$

The program works correctly. The grid size is rounded up with $\lceil N/32 \rceil$, and the kernel guard `if (i < N)` prevents threads beyond the vector end from accessing memory. The maximum absolute CPU/GPU error is zero.

5–6. Configuration and occupancy for $N = 263149$

This launch uses 8,224 blocks and $8{,}224 \times 32 = 263{,}168$ threads. Of these, 263,149 compute an output and 19 are inactive padding threads. The measured achieved occupancy is 33.67%.

N	Blocks	Threads/block	Launched threads	Active threads	Achieved occupancy
512	16	32	512	512	3.12%
263,149	8,224	32	263,168	263,149	33.67%

Table 1: Launch configurations and Nsight Compute achieved occupancy.

7. Timing breakdown for increasing vector lengths

Figure 1 shows 16 lengths from 512 to 16,777,216, including $N = 263149$. Host-to-device time covers both inputs; device-to-host time covers one output. The two copies grow approximately linearly with N and dominate at large sizes, while the simple addition kernel remains a small fraction of total time.

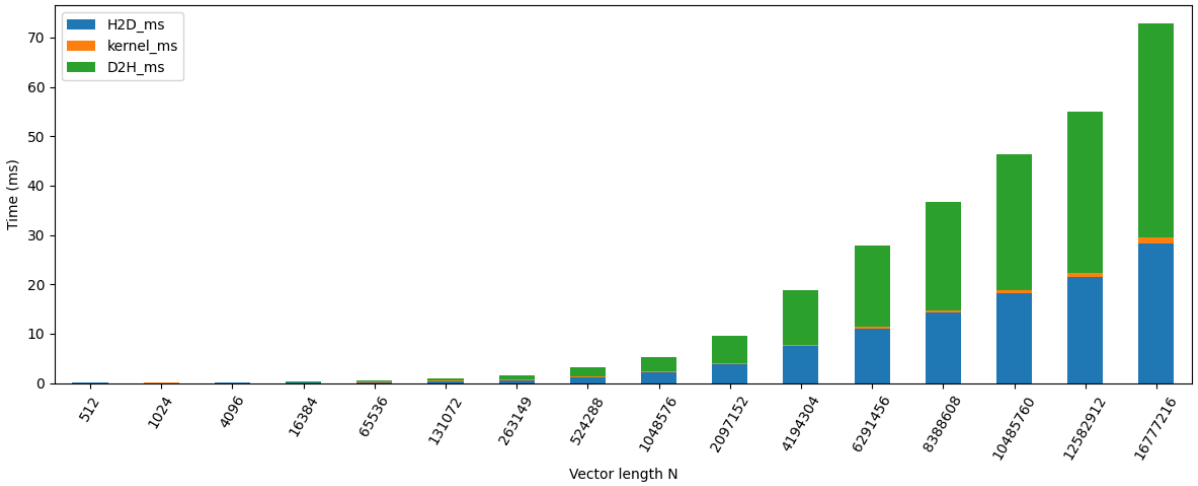


Figure 1: Q1 timing breakdown on the Tesla T4.

8. CPU/GPU numerical comparison

The outputs match exactly for all tested lengths: `max_error = 0`. Each result element is a single addition of the same two `float` operands on CPU and GPU, with no operation reordering or accumulated reduction error. Both therefore apply the same IEEE-754 rounding to the same exact sum. This does not imply that floating-point addition is exact in general; it means both implementations round identically here.

Question 2 — 2D Dense Matrix Multiplication

The program computes row-major $C(M \times P) = A(M \times N)B(N \times P)$. Each valid thread computes one output element. Blocks contain $16 \times 16 = 256$ threads, and the two-dimensional grid is

$$\text{grid} = (\lceil P/16 \rceil,\ \lceil M/16 \rceil).$$

The kernel uses no shared-memory tiling. The same templated implementation supports `double` and `float`.

1. Floating-point operations and global-memory reads

Each of the MP output elements requires N multiplications and N additions to accumulate the dot product from an initial zero. Thus,

$$\boxed{\text{FLOPs} = 2MNP}.$$

Each loop iteration reads one element of A and one element of B, so the kernel performs

$$\boxed{\text{global-memory reads} = 2MNP}$$

scalar element reads, or $2MNP\,\text{sizeof}(T)$ bytes before considering cache effects. It also writes MP output elements. These are source-level global accesses; hardware caches can reduce actual DRAM traffic.

2. Small matrices: 128×256 times 256×32

The grid is $(\lceil 32/16 \rceil, \lceil 128/16 \rceil) = (2, 8)$, giving 16 blocks and $16 \times 256 = 4{,}096$ launched threads. Every thread computes one result because $128 \times 32 = 4{,}096$. Nsight Compute reports **24.92% achieved occupancy** (7.97 achieved active warps per SM; theoretical occupancy 100%).

3. Large matrices: 1024×8191 times 8191×8197

The program still works. Two-dimensional launch bounds checks prevent out-of-range accesses, and 64-bit `size_t` allocation/index arithmetic avoids 32-bit byte-size overflow. The double run passed 81 deterministic CPU-reference samples with zero maximum absolute error.

The grid is $(513, 64)$, so there are $513 \times 64 = 32{,}832$ blocks and $32{,}832 \times 256 = 8{,}404{,}992$ launched threads. Only $MP = 1024 \times 8197 = 8{,}393{,}728$ threads compute results; the remaining 11,264 threads are padding in edge blocks. Nsight Compute reports **99.17% achieved occupancy** (31.74 achieved active warps per SM). The much larger launch keeps the SMs occupied, unlike the 16-block small case.

(M, N, P)	Grid	Blocks	Launched threads	Active threads	Achieved occ.
$(128, 256, 32)$	$(2, 8)$	16	4,096	4,096	24.92%
$(1024, 8191, 8197)$	$(513, 64)$	32,832	8,404,992	8,393,728	99.17%

Table 2: Q2 launch configurations and Nsight Compute achieved occupancy on the T4.

4. Timing breakdown with increasing matrix sizes

Figure 2 shows the `double` timings. For the cubic cases, copy sizes grow approximately as the square of the side length, whereas kernel work grows as its cube. Copy and launch overheads are visible for small matrices; from 1024^3 onward the kernel is the largest component and its share

increases rapidly. The naive inner loop repeatedly reads global memory and has no explicit data reuse through shared memory, which further increases kernel time.

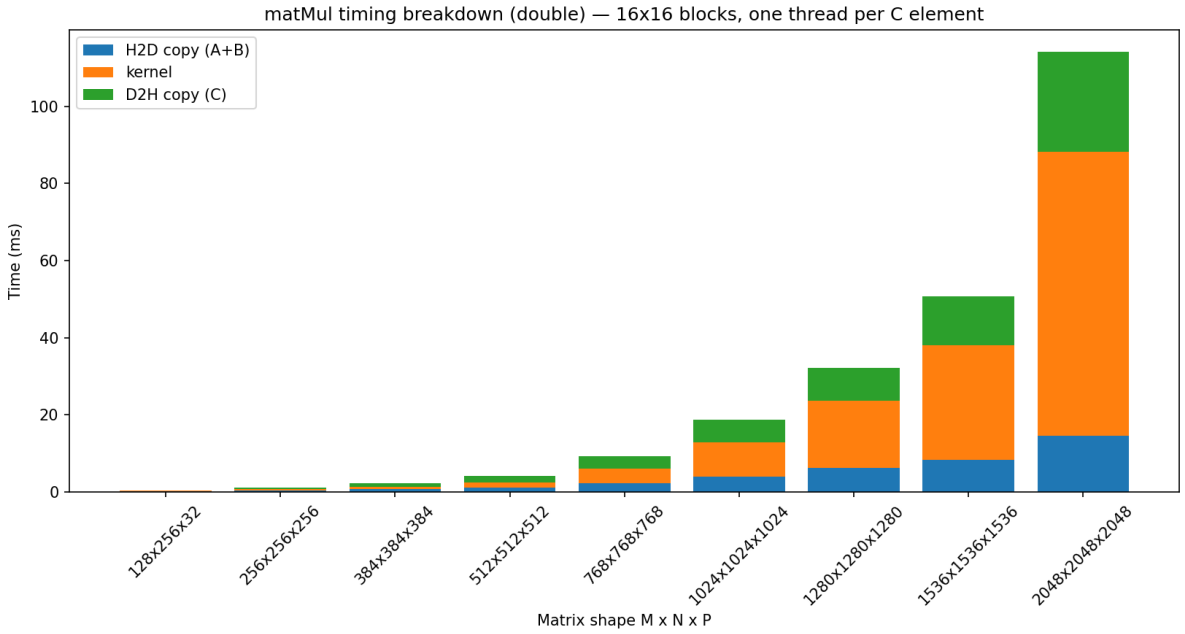


Figure 2: Q2 `double` timing breakdown on the Tesla T4.

5. Change from `double` to `float`

Figure 3 repeats the same sweep with `float`. At 2048^3, H2D time falls from 14.6677 to 7.8645 ms (1.87×) and D2H time from 25.9308 to 13.4261 ms (1.93×), close to the ideal halving of transfer time because each element is half as large. Kernel time falls from 73.4890 to 31.2176 ms, a larger 2.35× speedup. Both reduced memory traffic and the T4's much higher FP32 than FP64 arithmetic throughput contribute. Total measured time falls from 114.0875 to 52.5082 ms (2.17×). Smaller cases do not show the same ratio consistently because fixed launch and synchronization costs are significant.

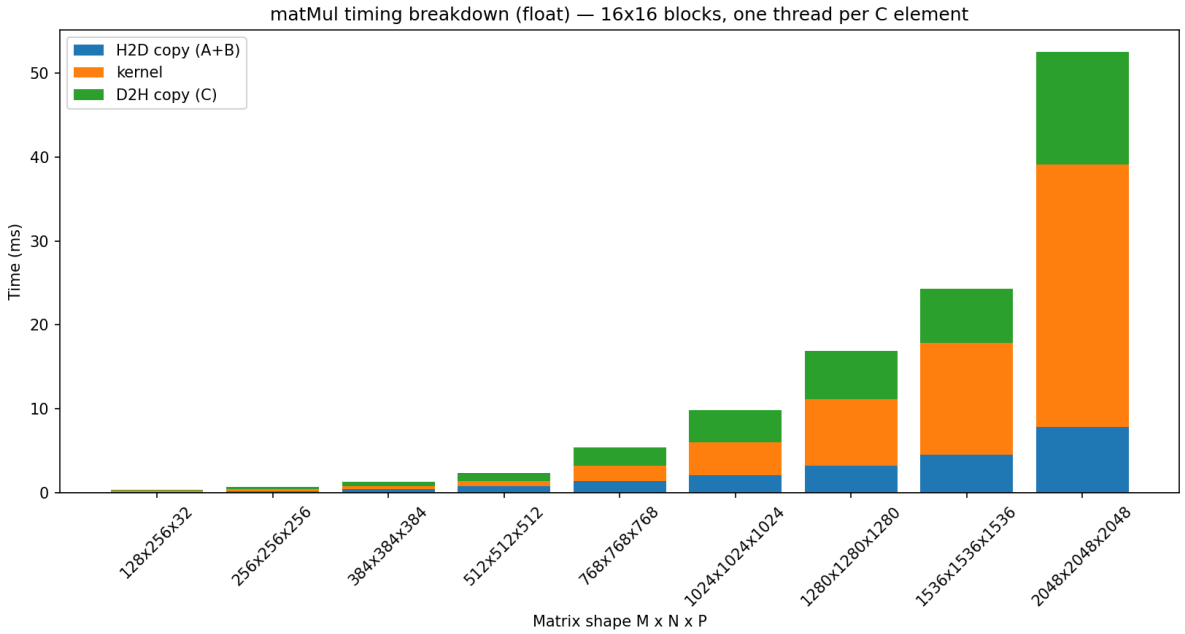


Figure 3: Q2 `float` timing breakdown on the Tesla T4.

Question 3 — Quantum Fourier Transformation

3.1 Troubleshooting and 24-qubit run

No troubleshooting was needed. The build commands from the assignment worked as given with compute capability 75, and the benchmark ran on the GPU backend.

```
[ ]
%%bash
cd /content/QuEST
./build/qft_naive 24 5 | tee /content/results/run_24q.txt

QFT benchmark: 24 qubits, 5 reps, GPU backend, 0.25 GiB state vector
  rep 0:  385.250 ms
  rep 1:  385.305 ms
  rep 2:  385.289 ms
  rep 3:  385.304 ms
  rep 4:  385.218 ms
  total prob    : 1.000000000000

  gates per QFT : 312
  best / mean   : 385.218 / 385.273 ms
  time per gate : 1234.674 us
```

Figure 4: Output of the 24-qubit QFT benchmark.

3.2 Memory at 28 qubits

Quantity	Value	How obtained
GPU memory (whole device)	4,203 MiB	`nvidia-smi memory.used`, polled during the run
GPU memory (this process)	4,200 MiB	`nvidia-smi` per-process `used_memory`
Process resident memory	179 MiB	`/usr/bin/time -v` and `VmHWM` in `/proc/PID/status`
System memory in use	1,308 MiB before; 1,515 MiB peak	`free -m`

Table 3: Measured memory use for the 28-qubit benchmark.

The run uses about 4.1 GiB of GPU memory. Almost all of it is the state vector: 2^{28} amplitudes at 16 bytes each equal 4,096 MiB. System memory use is small, about 180 MiB for the process.

3.3 Top three kernels at 24 qubits

The profile used the assignment's `nsys` command with `-trace=cuda,osrt -stats=true`, output prefix `qft_24q`, and benchmark arguments `24 5`.

Rank	Kernel	Total (ms)	Time	Instances	Avg. (ms)
1	`kernel_statevec_anyCtrlOneTargDiagMatr_sub<1>`	1,874.1	80.8%	1,656	1.132
2	`kernel_statevec_anyCtrlOneTargDenseMatr_subA<0>`	332.7	14.3%	144	2.311
3	`kernel_statevec_anyCtrlSwap_subA<0>`	96.4	4.2%	72	1.338

Table 4: Nsight Systems GPU kernel summary for 24 qubits.

The run has one warm-up and five timed QFTs, so each gate runs six times.

3.4 Kernels and quantum gates

Kernel	Gate	Per QFT (24 qubits)
`kernel_statevec_anyCtrlOneTargDiagMatr_sub`	`applyTwoQubitPhaseShift`	276
`kernel_statevec_anyCtrlOneTargDenseMatr_subA`	`applyHadamard`	24
`kernel_statevec_anyCtrlSwap_subA`	`applySwap`	12

Table 5: Mapping from QuEST CUDA kernels to QFT gates.

The phase gate is applied as a one-control, one-target diagonal matrix, the Hadamard as a dense 2×2 matrix, and the swap through `localiser_statevec_anyCtrlSwap`. The kernels are in `quest/src/gpu/gpu_kernels.cuh`. Six runs times 276, 24 and 12 give the instance counts in Table 4. The phase kernel dominates total time because it runs 276 times per QFT.

3.5 CUDA API calls on the CPU thread

A single CPU thread launches the kernels with `cudaLaunchKernel`, once per gate (312 per QFT). All 312 launches finish within 7.6 ms. The GPU needs 385 ms, so the CPU then waits in `cudaDeviceSynchronize`, which `syncQuESTEnv` calls.

3.6 Sixteen and 28 qubits

Qubits	QFT time, best / mean (ms)	Typical kernel: phase / Hadamard / swap
16	1.732 / 1.800	9.4 μs / 18.5 μs / 5.4 μs
24	384.96 / 385.00	1.13 ms / 2.31 ms / 1.18 ms
28	8,187.2 / 8,193.6	18.0 ms / 36.9 ms / 18.6 ms

Table 6: QFT benchmark times and typical kernel durations.

Kernel time scales with the state vector from 24 to 28 qubits: the state vector is 16 times larger and the kernels are about 16 times slower. At 16 qubits, kernels last only microseconds, similar to the approximately 5 μs launch cost, and the GPU is idle for about 15% of a QFT.

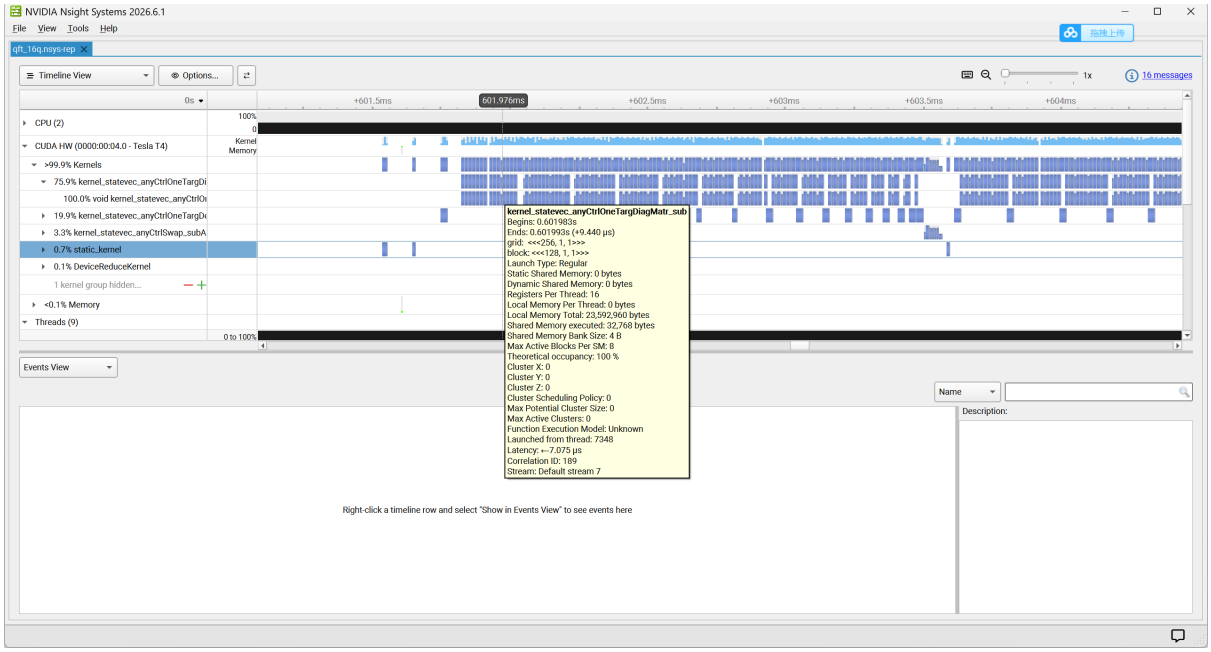


Figure 5: Nsight Systems timeline at 16 qubits. The tooltip shows one phase kernel lasting 9.440 μs.

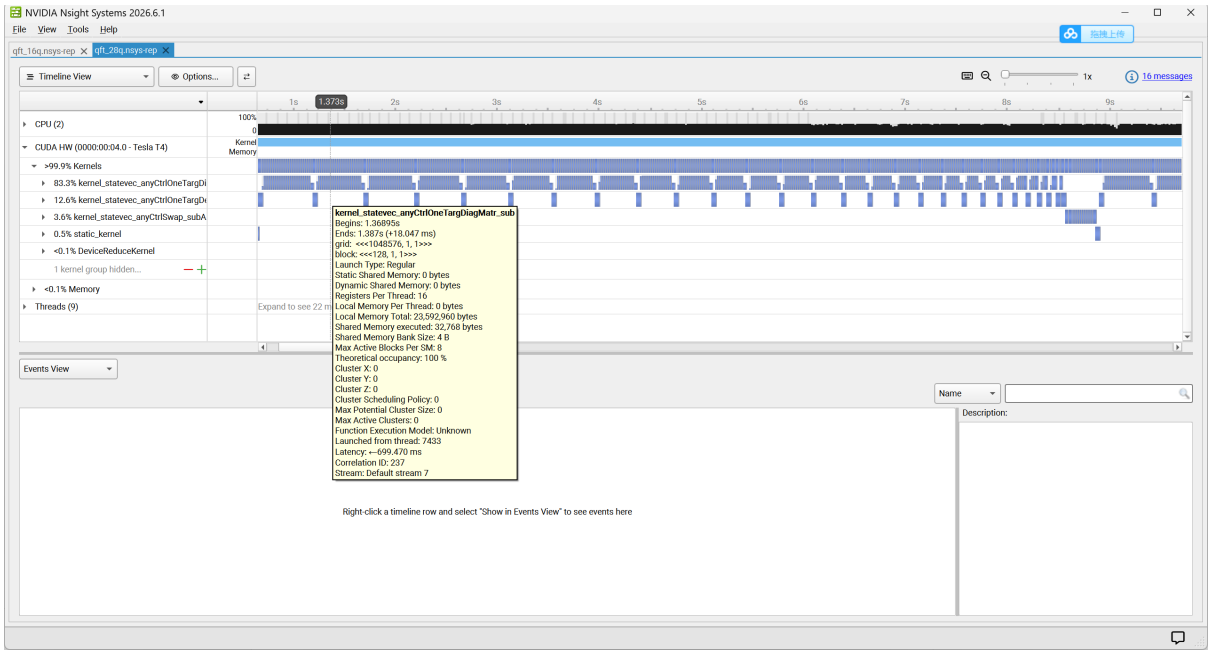


Figure 6: Nsight Systems timeline at 28 qubits. The tooltip shows the same phase kernel lasting 18.047 ms.